

IN31C-0673: Benchmarking Zstandard and Quantize Compression in NetCDF

Edward Hartnett^{1,2}, Hang Lei^{3,2}, Dusan Jovic^{4,2}

1. CIRES, University of Colorado, Boulder, CO, 2. NOAA/NWS/NCEP/EMC,College Park, Maryland 3. Science Applications International Corporation (SAIC), Reston VA 4. Lynker, Leesburg, VA

Summary

- Since netcdf-c-4.0 (2008), netCDF can create files with compressed data, using the zlib library.
- Once turned on, the compression/decompression is transparent to the user, but does add time to write and read the data.
- Since netcdf-c-4.7.4 (2020) compression has also worked with parallel I/O.
- Since netcdf-4.9.0 (2022) supports zstandard compression and lossy compression with the quantize feature.
- Using zstandard speeds compression and decompression.
- Using quantize turns on lossy compression, allowing much greater compression of data.

A History of Community Collaboration

Date	Feature	Impact	Collaborators
2016	NCO tools support new compression, bit-grooming.	New compression filters in post-processed data, demonstrates value of bit-grooming. (Geosci. Model Dev)	U. of California, Irvine
2019	Bit-shaving in GFS atmospheric history data.	Reduces size of ATM history file from 36 GB to 6 GB.	NOAA
2019	Creation of CCR project.	Allows testing of new compression filters in netCDF. (AMS)	U. of California, Irvine, Unidata/UCAR, NOAA
2020	NetCDF parallel I/O writing using compression.	Order of magnitude improvement in write time for GFSv16. (WMO WGNF Blue Book)	NOAA, Unidata/UCAR
2021	Test with UFS; Add quantize/zstandard support.	Demonstrated value to NOAA and the wider community. (AGU)	NOAA, Unidata/UCAR, NetCDF power users.
2022	netcdf-c-4.9.0.	Quantization/zstandard available to all. (EGU)	NOAA, U. of California, Irvine, Unidata/UCAR
2023	netcdf-c-4.9.2/ netcdf-fortran-4.6.1.	Increased ease-of-use for Fortran users.	NOAA, Unidata/UCAR, U. of California, Irvine,

Zstandard is Faster than Zlib

- Zstandard is lossless compression.
- Zstandard library is available on all platforms (Unix/Windows/Mac).
- Significantly faster and more compressive than zlib.
- Provides greater control (than zlib) of speed/compression trade-off.
- Very fast decompression.

Acknowledgements

Adding the new compression features was the work of many, including:

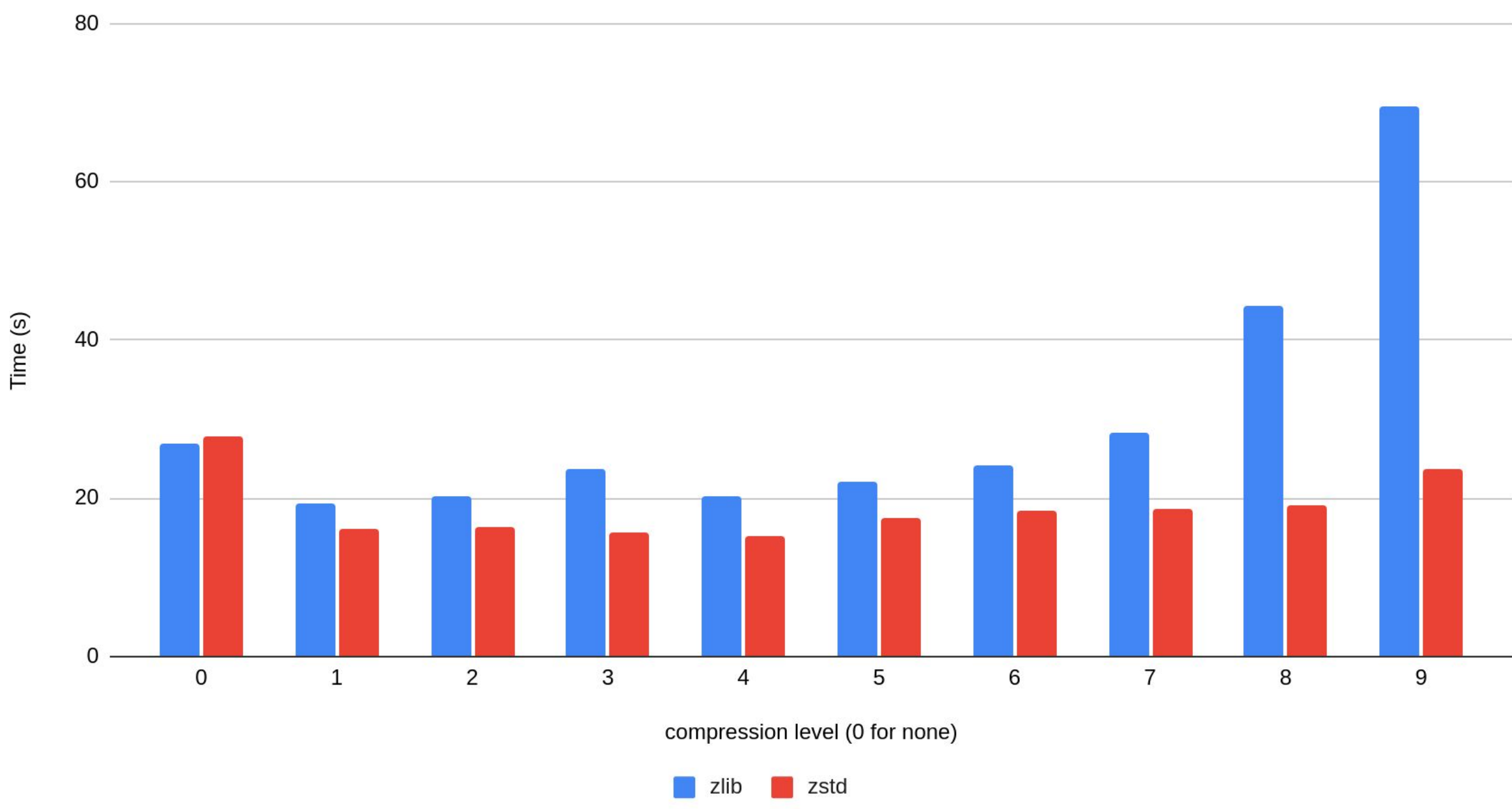
- Ward Fisher, Dennis Heimbigner, Unidata
- Charlie Zender, University of California, Irvine
- Edward Hartnett, Hang Lei, Brian Curtis, Alex Richert, Kyle Gerheiser, NOAA EMC.



NetCDF now supports faster compression with zstandard and also lossy compression.

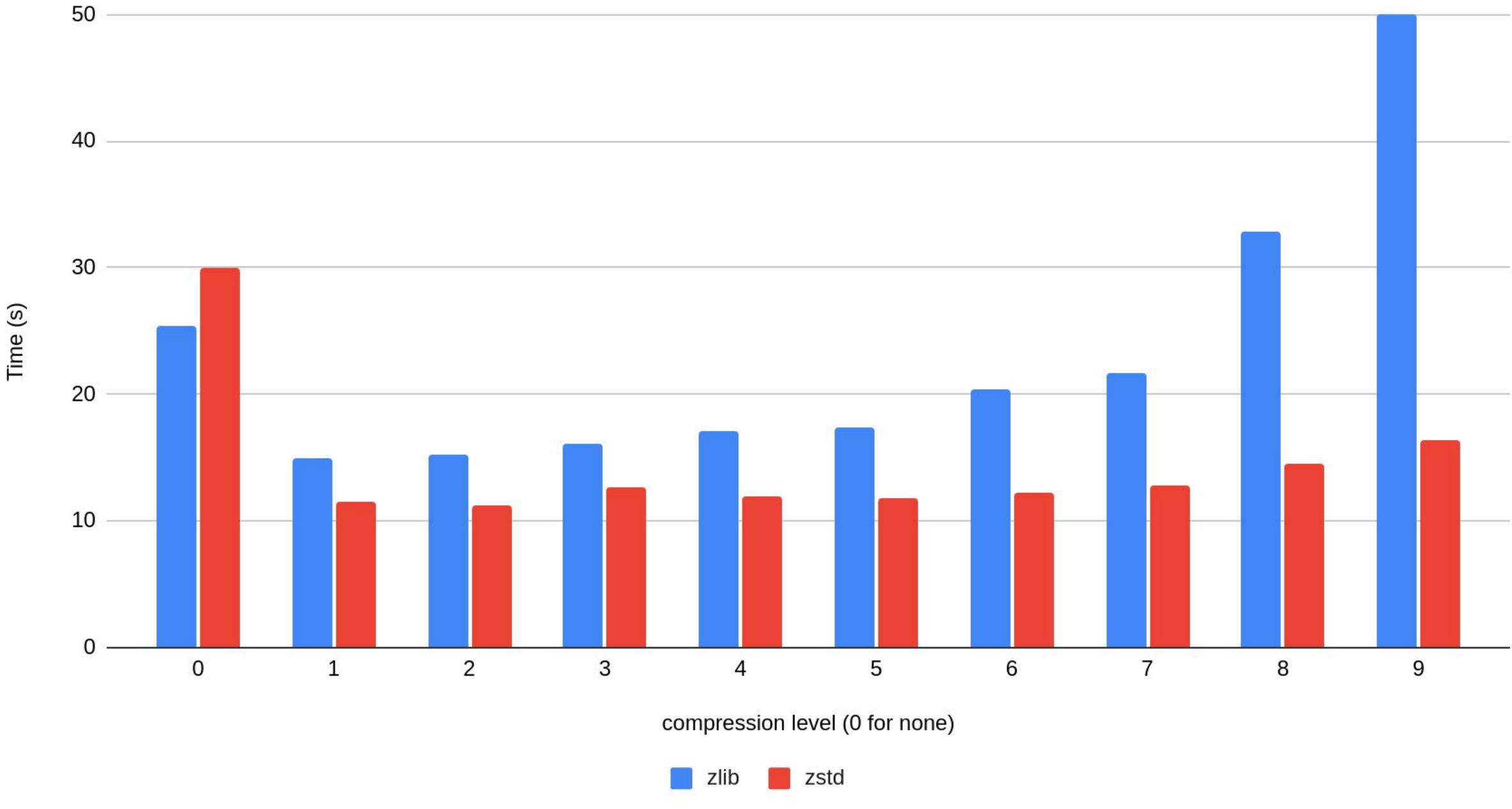
UFS ATM Run Time with Zlib and Zstandard Compression

Data written without quantize

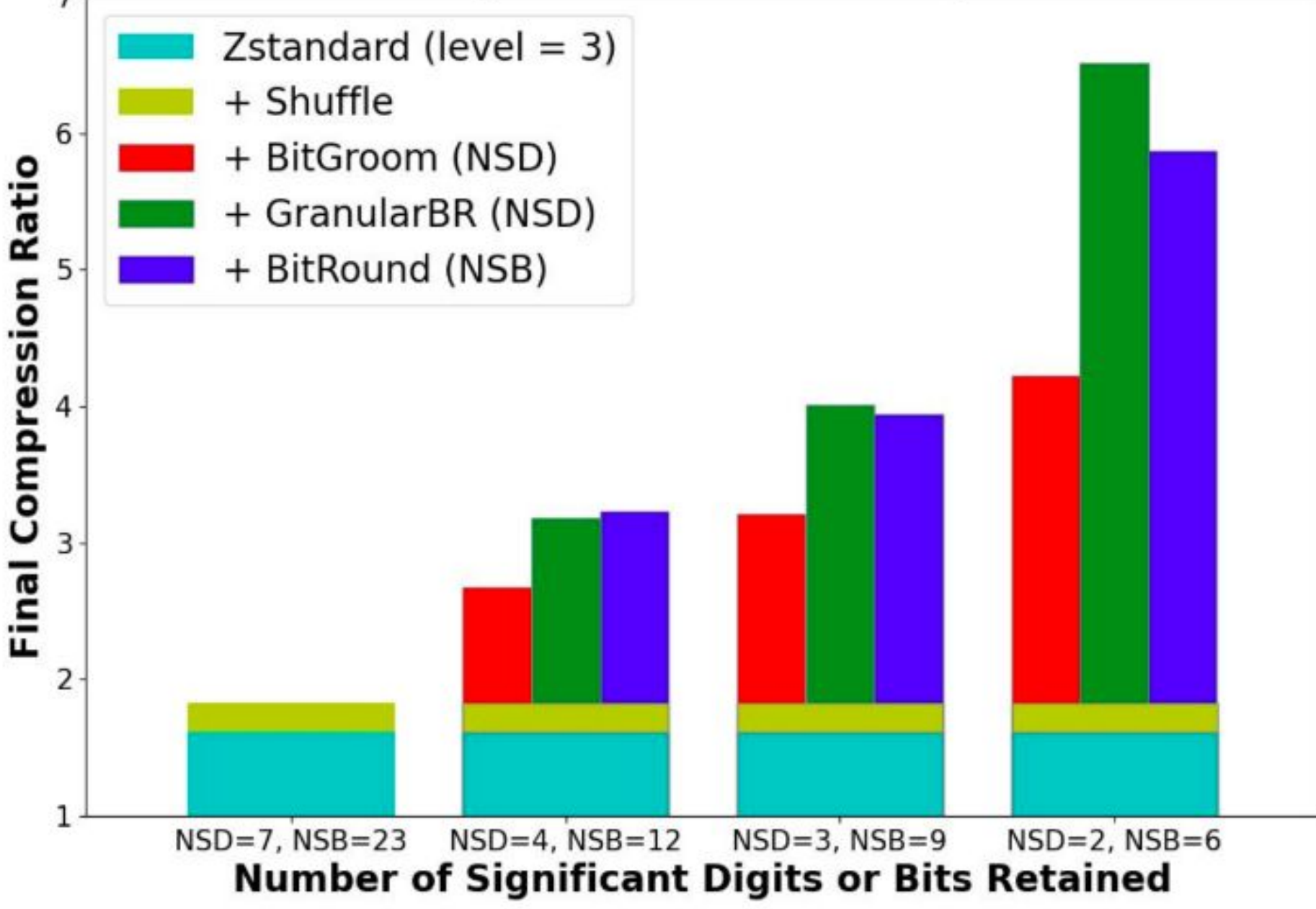


UFS ATM Run Time with Zlib and Zstandard Compression

Data written with quantize



Lossless & Lossy Contributions to Compression Ratio



Charlie Zender - Lossy compression: The netCDF implementation and towards encoding precision

Quantization Enables Lossy Compression

- Will only reduce data size if zlib/zstandard compression is turned on.
- Only for NC_FLOAT, NC_DOUBLE types.
- Fill values are not quantized.
- Quantize works and is tested with parallel I/O.
- Quantized data are fully backward-compatible, and can be read correctly by all versions of netCDF and netCDF-Java.
- An attribute is added to the data variable, recording the algorithm and the number of significant digits.

Lossy Compression of Pi

Sign	Exponent	Fraction (significand)	Decimal	Notes
0	10000000	10010010000111111011011	3.14159265	Exact
0	10000000	10010010000111111011011	3.14159265	NSD=8
0	10000000	10010010000111111011010	3.14159262	NSD=7
0	10000000	10010010000111111011000	3.14159203	NSD=6
0	10000000	10010010000111110000000	3.14158630	NSD=5
0	10000000	10010010000111000000000	3.14154053	NSD=4
0	10000000	10010010000000000000000	3.14062500	NSD=3
0	10000000	10010010000000000000000	3.14062500	NSD=2
0	10000000	10010000000000000000000	3.12500000	NSD=1

Figure 1: The value of Pi expressed as a 32-bit floating point number, with different levels of quantization applied, from Number of Significant Digits (NSD) equal to 8 (no quantization), to 1 (maximum quantization). The least significant bits of the significand are replaced with zeros, to the extent possible, while preserving the desired number of significant digits. In this example the Bit Grooming quantization algorithm is used.

Hartnett, Zender, EGU22-13259, [Adding Quantization to the NetCDF C and Fortran Libraries to Enable Lossy Compression](#)

Quantize Example in C

```
/* Create two variables, one float, one double. Quantization
 * may only be applied to floating point data. */
if (nc_def_var(ncid, "var1", NC_FLOAT, NDIM1, &dimid, &varid1)) ERR;
if (nc_def_var(ncid, "var2", NC_DOUBLE, NDIM1, &dimid, &varid2)) ERR;

/* Set up quantization. This will not make the data any
 * smaller, unless compression is also turned on. In this
 * case, we will set 3 significant digits. */
if (nc_def_var_quantize(ncid, varid1, NC_QUANTIZE_BITGROOM, NSD_3))
ERR;
if (nc_def_var_quantize(ncid, varid2, NC_QUANTIZE_BITGROOM, NSD_3))
ERR;

/* Set up zlib compression. This will work better because the
 * data are quantized, yielding a smaller output file. We will
 * set compression level to 1, which is usually the best
 * choice. */
if (nc_def_var_deflate(ncid, varid1, 0, 1, 1)) ERR;
if (nc_def_var_deflate(ncid, varid2, 0, 1, 1)) ERR;
```

Quantize Example in Fortran 90

```
! Define some variables.
call check(nf90_def_var(ncid, VAR1_NAME, NF90_FLOAT, dimids,
varid1&
&, deflate_level = DEFLATE_LEVEL, quantize_mode =&
& nf90_quantize_bitgroom, nsd = 3))
call check(nf90_def_var(ncid, VAR2_NAME, NF90_DOUBLE, dimids,&
& varid2, contiguous = .TRUE., quantize_mode =&
& nf90_quantize_bitgroom, nsd = 3))
```

Quantize Example in Fortran 77

```
C Create some variables.
do x = 1, NVARS
retval = nf_def_var(ncid, var_name(x), var_type(x), NDIM1,
$ dimids, varid(x))
if (retval .ne. nf_noerr) stop 3

C Turn on quantize.
retval = nf_def_var_quantize(ncid, varid(x),
$ NF_QUANTIZE_BITGROOM, NSD_3)
if (retval .ne. nf_noerr) stop 3

C Turn on zlib compression.
retval = nf_def_var_deflate(ncid, varid(x), 0, 1, 1)
if (retval .ne. nf_noerr) stop 3
end do
```

